Chapter 16. Data Management

By John Tobin and Victoria Wang
with Garrett Holthaus

Data management is the process of managing the data that your users and the company have entrusted to you, ethically and legally, while enabling your company to deliver services and value to your users. Data is one of a company's most valuable assets and needs to be treated with appropriate safeguards and respect.

Many companies don't think sufficiently about data management until something goes wrong, but like closing the barn door after the horse has bolted, it's too late to start caring about it then. Failures in data management are categorically different from other types of failures because remediation is so much harder, and frequently impossible. Data loss is permanent if you don't have working backups, and data security breaches can't be undone once the data has been exfiltrated. The trust and reputational damage caused by data loss or a data security breach can be immense and take years to recover from, if recovery is even possible. Real-world examples of poor data management are numerous, so instead we'll focus on an example of a company that handled data management exceptionally well. On 9/11, Aon's offices in the World Trade Center were destroyed, but the company's excellent data management plan allowed it to start operating again within a few hours and without any data loss.

This chapter breaks down how to build a robust data strategy, using real-world insights from Google SREs. It's tempting to jump directly into the technical stuff, but data management must always serve business and legal needs first. The topics we will cover include:

- Aligning data management with organizational and legal requirements
- Categorizing data to apply the right management strategies
- Protecting data soundness and safeguarding it from threats
- Identifying risks, and the engineering techniques used to mitigate them
- Defining the core SRE metrics needed to measure success
- Utilizing replication, backups, and restores to keep data accessible

- Crafting actionable backup plans, data recovery processes, and recovery policies, while maintaining low costs

This chapter is authored by SREs running Bigtable, one of Google's largest infrastructure services, storing 10 exabytes of data and receiving 7 billion queries per second, so we'll touch on Bigtable as we discuss the various facets of data management.

Legal and Ethical Considerations

When dealing with data, we first need to think about the business, legal, and ethical requirements. Once we know those, we can decide on technical solutions. Some of these concerns depend on the industry, country, and/or global region. Some concerns depend on the sensitivity of the data—e.g., whether it includes personally identifiable information (PII). Others, particularly the ethical considerations (see "Ethical Concerns"), are directly influenced by your company's values and culture.

Legal Concerns

We are not lawyers. As such, we won't try to give you any legal advice, other than suggesting that you should find out what the legal requirements are in the jurisdictions where your data originates, is physically stored, and is processed (as they're often different) and ensure you comply with them. Your organization should engage legal counsel with expertise in this area. We'll give some brief pointers about things you should consider here, but by no means should this be considered a comprehensive list.

Age of your users

Some jurisdictions have laws mandating different handling of user data relating to minors and adults. In many jurisdictions, you must minimize data collected from minors to only the data that is essential to provide the service, offer clear and plain language for the terms and conditions to minors, restrict processing of data related to minors, and refrain from building a behavioral profile for a minor.

For example, the Children's Online Privacy Protection Act (COPPA) in the US applies if your website or app provides services to children under 13. Before a company can collect any data from a child under the age of 13, verifiable consent must be obtained from an adult.

Data retention

Many jurisdictions have laws about data retention, such as not storing data after it has been processed, or ensuring full deletion of data within a limited period of time after the user has requested that you delete it. You will need a clear understanding of the different layers of data processing and storage to allocate sufficient time for deletions. Full deletion includes the expiry of backups, which may limit the maximum retention period for your backups. You will also need to monitor the age of your data and the progress of deletion at each layer to detect violations, and budget engineering time to investigate and fix the causes of violations. You may even need to make changes to your product to be fully compliant with data retention regulations. For example, if your product has a sharing feature and supports reshares, when a user deletes shared data, does your product also delete the reshared data? You should check with your legal team whether this is required, as it might depend on the legal jurisdiction. Audits can be expensive, particularly for smaller companies, so it's important to design your systems and processes to demonstrate compliance to auditors as easily and cheaply as possible.

Some techniques that can be helpful include:

Supporting soft and hard deletes

Depending on your requirements and data retention policies, you might want to support two deletion options. With a soft delete, the deleted item is marked as deleted so that it is no longer returned in searches, but it remains physically present in the database or storage system. This makes it much easier to restore the item if necessary. With a hard delete, all data associated with the deleted item is permanently removed from the storage system, making recovery impossible.

Setting a time to live (TTL)

Some storage systems support setting a TTL on individual data objects when they are created or updated, enabling the system to automatically clean them up when the TTL expires. Ideally, the storage system will implement this as a soft delete so that the item can be recovered later if necessary. Ensure you know how to clear or change the TTL to prevent deletion of data with an incorrect TTL. It's also a good idea to have a mechanism to disable TTL deletion entirely. You will want to measure how timely the deletions are, as well as their impact on your service—i.e., does your application depend on this data?

A relevant regulation you may be familiar with is the European Union's General Data Protection Regulation (GDPR), which aims to give individuals greater control over how their personal data is used. The GDPR sets out detailed rules governing how organiza-

tions may collect, process, store, and share this data. Failure to comply can result in significant fines—up to €20 million or 4% of the organization's global annual revenue, whichever is higher—as well as reputational damage.

CASE STUDY: DATA RETENTION IN GOOGLE'S BIGTABLE

Google's internal Bigtable service provides two SLOs for data deletion: the *fast* SLO is what users can expect when everything is working normally, and the *eventual* SLO is what Bigtable guarantees even when there are disruptions (e.g., if a user has underprovisioned Bigtable for the load it is receiving). Delivering on the eventual SLO requires Bigtable to prioritize deletion over user-facing traffic, a change in behavior that was implemented specifically for GDPR compliance. Providing these SLOs allows upstream application developers to add these expectations into their products to help ensure they are achieving end-to-end compliance. The SLOs were determined by the Bigtable SRE team, in coordination with the teams running the underlying data storage systems and stakeholders from the largest applications using Bigtable.

In the original design, Bigtable stored a bit of text from the user data as part of its metadata. Significant changes were made to Bigtable's bookkeeping to ensure that this piece of user data is expunged in a timely manner.

Data residency

Many jurisdictions have laws about data residency. You need to consider where the data originated, where it is stored and processed, and how it is handled in emergency situations. Google Cloud has a "hurricane policy" to ensure it doesn't violate data residency commitments for data that customers have indicated is subject to data residency laws. It instructs SREs that, when dealing with regionally restricted data, they must not attempt to move it, as doing so could violate the law for some customers. Those customers would prefer unrecoverable data loss to breaking the law, so we accept data loss under these compliance conditions. For data that's not subject to data residency requirements, however, we will of course do our utmost to move it if that's what's required to keep it safe. See "Data Security" for more on why securing data is an important aspect of data management.

For example, some digital-forward countries have rules restricting sensitive government data from traveling beyond their borders. From an infrastructure perspective, in the last decade, digital sovereignty has emerged as a focus. In Europe, multiple countries (e.g., France, Germany, and Italy) have started operating sovereign clouds that are controlled

at the country level rather than by the hyperscalers themselves. The goal is to mitigate geopolitical risks associated with data governance and control.

Restrictions on usage of data

Many jurisdictions have restrictions on using user data for a different purpose than the purpose for which it was originally collected. If you have products that are quite different from one another, you will need to keep their data separate and record the purpose for which it was obtained. Depending on how the data is structured, it may need restructuring and separation, or applications may need to migrate from direct data access to access through a proxy that can make policy decisions about whether the access should be allowed. Both Google's Cloud and internal storage and database systems have complex policy-based controls that authorize access. These policies are defined by the users, because they understand the data they're storing.

One provision of the EU's Digital Markets Act (DMA) restricts whether different divisions of a company are allowed to share data. Users of YouTube Music provide their music preferences to receive better recommendations, but those preferences are not used by Google Ads unless the user has explicitly given permission. Similarly, the US Health Insurance Portability and Accountability Act (HIPAA) restricts access to patient data unless it is authorized by the patient.

Ethical Concerns

You may have heard of *Dungeons & Dragons* character alignments (see Figure 16-1), an idea that has become a popular tic-tac-toe-style meme format. The concept categorizes a character's ethical outlook along two axes: good versus evil and lawful versus chaotic. Thus, for example, it's possible for a character who never breaks the law to be *Lawful Good*, *Lawful Neutral*, or *Lawful Evil*:

- A Lawful Good character doesn't just obey the law; they make altruistic decisions and will make sacrifices to help others.
- A Lawful Neutral character won't actively exploit people, but won't make sacrifices to help people either.
- A Lawful Evil character obeys the law but will always make choices to benefit themselves, regardless of the cost to others.

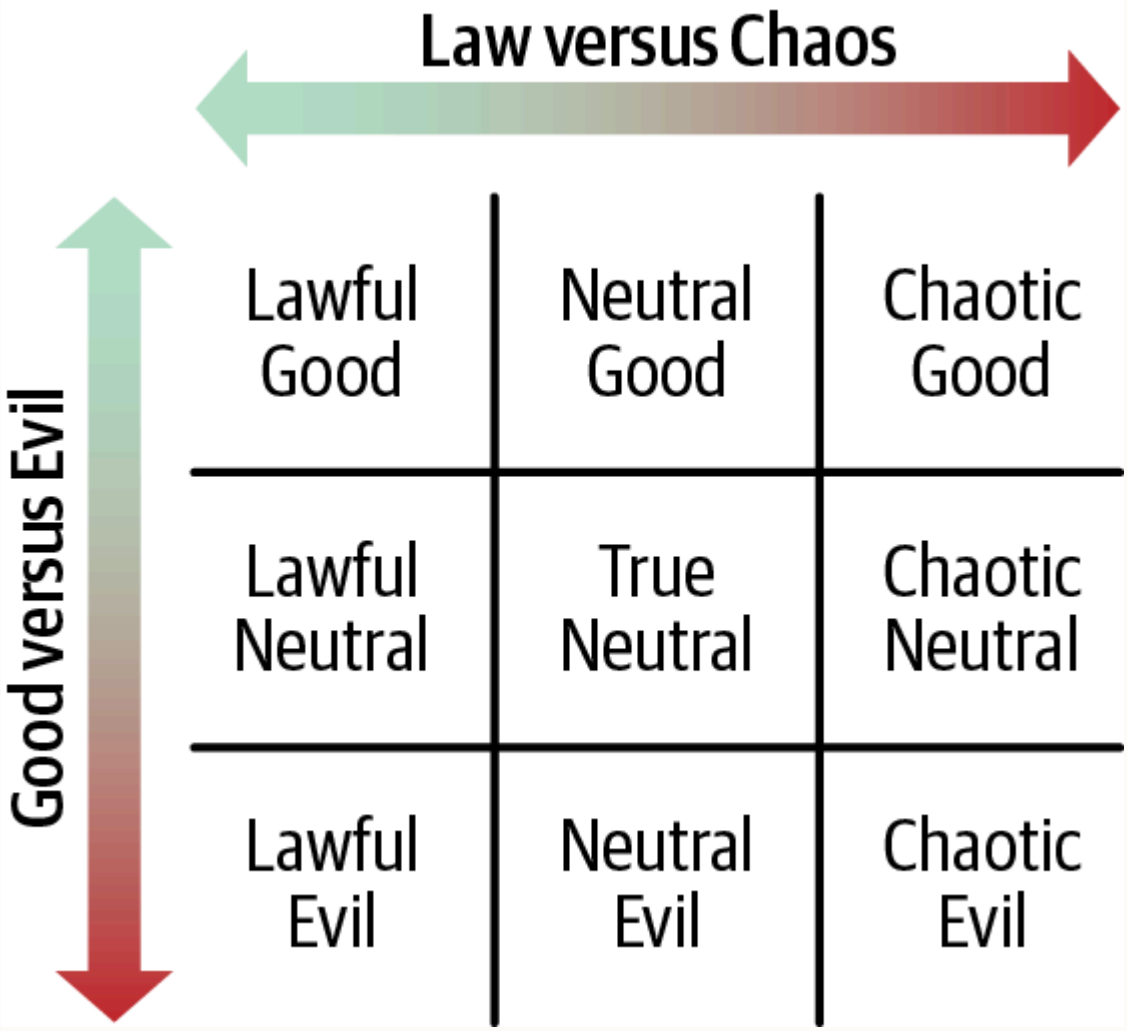


Figure 16-1. The alignment chart from Dungeons & Dragons (image adapted from Wikimedia)

Strive to be Lawful Good. Definitely don't be Lawful Evil. There will likely be uses for the data you've been entrusted with that, while legal, are not ethical. If you do something the user didn't explicitly ask for but you think it's useful for them, it's getting risky (so consider asking them first!). If you do something the user didn't ask for and it's only benefiting you, consider how they might react if they found out how their data was being used. These situations have led to negative press, loss of user trust, and impacted brand reputation.

Data Classification

Data ranges from being relatively inconsequential to encompassing highly sensitive information, such as financial records or medical histories. The value of the data affects the level of care you should take with it; losing someone's high scores from a game is far less damaging than losing their financial data, which could have serious real-world consequences. Although the true value of a piece of data can only be determined by its owner, some reasonable generalizations can be made: financial data, email, and photos, for example, are all inherently valuable. If in doubt, consider the data valuable until you have time to classify it correctly.

The data you're responsible for can be classified along multiple independent dimensions, each of which has an impact on how it should be managed. From a business perspective, different kinds of data have different requirements. Some things to consider for each kind of data at your company are:

- How long should the data be stored?
- Who can access it?
- What should your backup strategy be? (Backups, snapshots, replication, and more are covered in detail later in the chapter.)
- What are the incident response procedures for restoring it?

Let's take a look at some of those classifications. This isn't an exhaustive list; it's intended to get you thinking about data classification in general.

Original Data Versus Derived Data

Derived data is created from original data through some transformation process. This is analogous to binaries and source code: the source code is your original data, whereas binaries are derived from the source code. Original data requires a higher standard of care than derived data because if it is lost it is unrecoverable. Source code is a good example again, but so are photos, user reviews, emails, and generally any kind of user-generated content. The aggregated ratings and recommendations derived from user reviews can be re-created as long as you have the user reviews as input. Backups are especially important for original data (see "Backups and Restores").

Some derived data could technically be re-created, but doing so would be too expensive, so it needs to be treated similarly to original data. The Google Search index is an example: the original data is available (and the index is always being refreshed), so the index could be re-created from it. However, re-creating the index would take substantial time and resources, and Google Search would be unavailable for some of that time, so the index is treated with the same care as the original data.

Identifiable Versus Nonidentifiable Data

Identifiable data is any data that can be linked back to a specific person, business, organization, or other entity. Nonidentifiable data is identifiable data that has been anonymized. Identifiable data typically has stricter legal requirements for how it is managed than nonidentifiable data. Anonymization is extremely hard to do correctly, and the obvious approaches of hashing identifiers or simply removing identifiers are insufficient. Hashed identifiers can still be linked across pieces of data, and data with identifiers re-

moved can often be deanonymized by joining it with other data sources. The overarching guideline used inside Google is "A piece of data must be generalized across multiple data subjects to be considered anonymous." Google provides a public guide to data anonymization.

High-Value Personal Data

Some personal data is intrinsically more valuable to the user than other personal data. Common examples include:

- Passwords, private keys, and passphrases
- Date of birth, home address, and government identifiers (Social Security number, taxpayer identification number, etc.)
- Sensitive personal information such as ethnicity, sexual orientation, religious affiliation, and gender

Before collecting high-value personal information, ask yourself whether you absolutely require it. Do you have a clear justification for gathering sensitive user data? Imagine if this data were stolen, leaked, or sold. How would you explain to your users your decision to collect it and the storage architecture you store it in? More importantly, how would you explain it to your legal counsel? Data breaches are unfortunately common and expensive.[1]

If you have convinced yourself there is a good reason to obtain and store this information, you should seek legal advice to understand the legal requirements around managing the data. Some companies have dedicated privacy specialists. If yours does, consult them for a privacy review. The business value of having this information might not justify the business or legal risk.

If your company already has sensitive or high-value data, be sure to request a legal review before applying it to a new use case. Consider whether the risks to the business and the users are justified by the benefits, whether the users agreed to this use of their information, and whether the processing you do meets the required standards.

Here be dragons—be very wary of poking them.

The Hierarchy of Data Needs

Now that you've thought about legal and ethical considerations and classified your data, you're ready to manage that data.

The motto of the Google Storage team is "You, and only you, can always read your data." There's a lot to unpack in this pithy statement. Briefly:

- "You, and only you" is the data security aspect (see "Data Security"). Your data must be accessible only to you and those you explicitly share it with.
- "...can always read your data" covers two aspects: data integrity (see "Data Integrity") and data availability (see "Data Availability"). Your data must not be corrupted in any way; it must be returned to you exactly as it was written. Your data must also be available to you when you try to access it.

Although the motto doesn't directly cover data efficiency (see "Data Efficiency"), Google's storage products ensure that you can access your data at a reasonable cost, because high access costs prevent actual usage.

These four elements form the hierarchy of data needs, modeled on Maslow's famous hierarchy of needs (Figure 16-2).

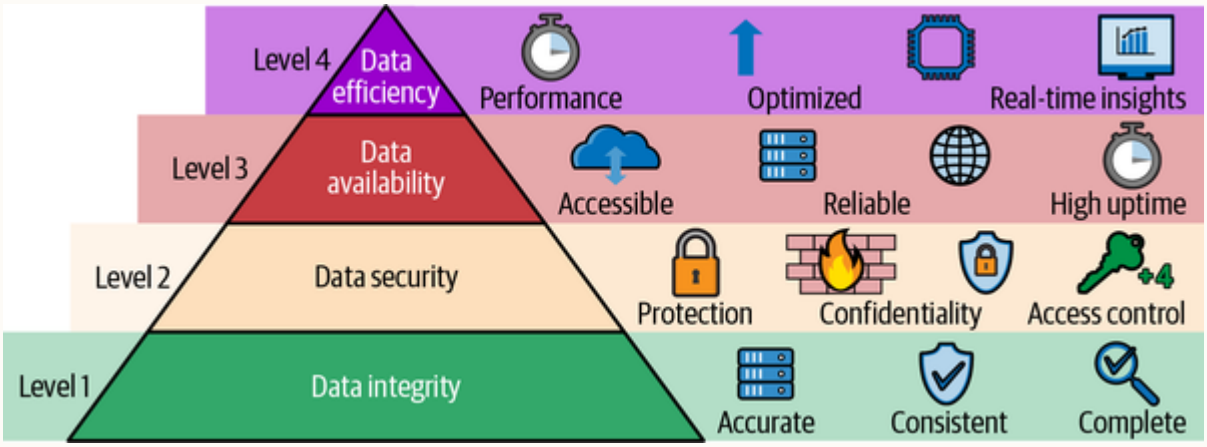


Figure 16-2. The hierarchy of data needs

Data Integrity

The first layer in our pyramid is data integrity: the fundamental guarantee that retrieved information matches exactly what was originally written, ensuring that data is neither corrupted nor lost over time. Often summarized by the principle "what you read is what you wrote," it serves as a critical measure of the accuracy and accessibility of the data

stores used to provide a service. To maintain this standard, a robust data management strategy must coordinate several distinct technical mechanisms:

- Data resiliency through storage redundancy and fault tolerance to survive hardware failures
- Checksumming to proactively detect silent data corruption (SDC)
- Appropriate implementation choices such as append-only logs and multiversion concurrency control (MVCC) to minimize the risk of corruption
- Validation processes to identify and repair logical inconsistencies

Together, these interconnected components ensure that user data remains trustworthy and consistent throughout its lifecycle.

Data resiliency

Data must be stored in a way that is resilient to hardware and software failures. This requires redundancy across failure domains, including multiple disks or SSDs, servers, and racks. However, this still leaves your data in a single region, making it vulnerable to a regional failure. It is highly recommended that you keep copies of your data in multiple regions (while respecting legal and regulatory requirements) to protect against the unavailability or loss of an entire region. Replication policies (see "Replication") should be chosen carefully to balance resilience against costs.

Consistency checks

Internal consistency checks can prevent bad data from being written. SQL databases provide foreign key constraints to enforce referential integrity, `UNIQUE` constraints to prevent duplicates, and `CHECK` constraints to prevent invalid values from being written to a row. These are a valuable defense against application-level bugs that might result in storing bad data, although the application must be written to handle the errors returned by the database. Other types of higher-level storage systems may offer similar protections.

Checksums

To protect against data integrity failures, Google uses checksums at multiple layers: the application layer, every storage layer underneath, and the RPC layers in between to cover data in flight. Calculating and verifying all those checksums is expensive, but it's how we identify when data is corrupted.

Silent data corruption (SDC) is a problem at scale: hardware failures can cause bit-flips in RAM or incorrect calculations on CPUs, corrupting data. Although these hardware failures are rare, at a large enough scale they will happen. It's essential that SDC be detected as soon as possible to give the best chance of remediation. If an RPC can fail because SDC is detected, there's a chance that the sender can retry the operation; however, if SDC is detected hours or days later, the data is far more likely to be unrecoverable.

CASE STUDY: FILE CORRUPTED DURING REPLICATION

Why do we have so many checksums? Because of collisions and the fact that corruption can happen in the CPU or RAM of any machine that's processing the data, or on the network while it's in transit. If checksums were computed and checked only by the last server in the chain, and data was corrupted before reaching that server, then the server would calculate the checksum of corrupted data.

Once upon a time, an internal user opened a help request for the SREs managing the Google File System (GFS). The user reported that for a single file, approximately one read in three would fail to pass their application-level checksums. This had the hallmarks of a single corrupt chunk, so the SRE handling the help request computed checksums for the three chunk copies. (Data corruption is an acceptable reason to touch user data, and the SRE can perform such a check without looking at the raw data.) They deleted the chunk that was different and let GFS notice that a chunk was missing and re-replicate it.

However, the client reported that the problem was still occurring, and when the SRE again compared the checksums of the three chunks, they found that one of them was once again different, despite being copied from a good server. No other problems were reported for that machine: it was serving other chunks correctly and running multiple services correctly. They engaged onsite hardware operations, who swapped out the RAM and CPUs on the destination server, the RAM and CPUs on the source servers, and the network switches between the servers—but the chunk was still corrupted when transferred to the destination server.

Finally, they replaced the Cat5 cable between the destination server and the switch, and the chunk was transferred correctly. It turned out that the network cable was corrupting the data in transit in a way that passed the TCP-, RPC-, and GFS-layer checksums but was finally detected by the application-layer checksums. This illustrates why we have so many layers of checksums: at a large enough scale, one-in-a-trillion events will eventually happen. This rare event helped to create a company-wide culture of verifying data.

Bigtable is one of Google's largest infrastructure services, so it has invested significantly in SDC prevention and detection, with checksums being the primary technique. This extensive use of checksumming has reduced the rate of SDC in Bigtable to roughly one minor event (e.g., a single unrecoverable file affecting a small number of tablets) per year. We have a data recovery policy (see "Creating a Data Recovery Policy") and data recovery tooling for use in those events.

Application-level verification

Application-level verification is very helpful in detecting application-level bugs that cause inconsistent data to be stored. Inconsistencies can be introduced in multiple ways. Here is a partial list to consider:

- Having two sources of truth for one piece of data that needs to stay in sync, where neither can be derived from the other
- Data that has been denormalized for performance reasons
- Race conditions between processes operating independently on the same data
- Updates across multiple data sources that cannot be wrapped in a single transaction
- A task crashing part way through an update
- Decisions made based on updates that haven't completely replicated

Sometimes these inconsistencies and the resulting reconciliations are expected because of an engineering trade-off—e.g., queueing updates for later actuation rather than blocking on applying the full update when the user requests it. We distinguish between *reconciliations* and *repairs* to differentiate expected inconsistencies from unexpected inconsistencies. Expected inconsistencies are part of the system design, whereas unexpected inconsistencies are bugs that should be fixed.

Ideally, the repair process for unexpected inconsistencies should capture enough debugging information to enable application developers to debug, reproduce, and fix the bug. Silently repairing unexpected inconsistencies can hide bugs for a long time, and sometimes the inconsistencies can build up to the point where they can't be repaired. Repair operations should be logged and monitored with alerts that fire when the number of repairs exceeds a certain threshold, which is decided by the business. In some cases, that threshold is one because any repair indicates a serious problem. In others a low background level of repairs may be acceptable, with an alert triggered if the number increases, indicating that a new bug may have been introduced.

Consider Google Play's architecture for handling user migrations between family groups. Because cross-group migration is a rare change that impacts multiple downstream ser-

vices, the platform records the change and applies it asynchronously rather than executing it synchronously. If a single dependency update fails, the next run of the reconciliation pipeline automatically catches and applies the missed state change. The transient delay is an acceptable, well-monitored engineering trade-off that minimizes overall system blast radius.

Data Security

As Figure 16-2 shows, the next layer of the pyramid is data security. Like data integrity, it is a foundational concern. Users care deeply about their data not being accessible to people or processes that they haven't agreed can access it. (See Cisco's 2024 Consumer Privacy Survey, where 75% of respondents said "I will not purchase from organizations I don't trust with my data."). This includes staff in the company providing the service to users, and staff at the underlying infrastructure providers used by the company. Every effort should be made to reduce and limit access to user data. Where exceptional access to data is essential—e.g., to debug a server that crashes when it tries to load data—the access granted should be as limited as possible (for example, access to a single account rather than all accounts, and only for the time required for the investigation). All exceptional accesses should be logged for future analysis, ideally to a system that is not under the control of the person who is accessing the user data, and without a way to skip this logging.

Data access

For each type of data you store, you should ask who can access that data and for how long. The core security principle governing access is the principle of least access (PoLA), granting each user, service, and system the minimum necessary privileges to access the minimum data required. If you are storing your data in a cloud provider, customer-managed encryption keys allow you to ensure that your cloud provider can't access your data. If you want to protect your data from a single entity attempting malicious actions (or just plain old human error), you can implement multiparty approval so that a second pair of eyes verifies that the operation won't result in unwanted consequences. Different data or operations can have different requirements; for example, people's names might be widely accessible but addresses kept tightly restricted.

You might have seen data classifications such as *confidential*, *internal-only*, *public*, *PII*, etc. You wouldn't want your company's website to be displaying data that is internal-only, and you might not want confidential information being shared across divisions for legal reasons. Classifying your data and putting in place rules for data access is impor-

tant to ensure the data you intend to share is not being shared on an unintended platform or with unintended parties.

Test data

Engineers need data to test with, and the easiest option is often to test with production data. People generally start testing with production data because their live dataset is small and easily managed. In a fast-paced startup, they may believe there isn't time to create test data and keep it up to date. While this is understandable, it's not acceptable or sustainable in the long term: at some point, significant effort needs to be invested in creating test data and test environments.[2] In addition, cultural expectations must be set to maintain the test data through changes and to create new test data as an essential part of building new features. Otherwise, engineers retain far too much access to user data, and you're only one bug away from accidentally running `delete from ...`. There are, sadly, many examples of data loss caused by an engineer mistakenly running test code or commands against production.

At Google, data is managed by dedicated Storage and Database teams. These teams have entirely separate test instances, frequently called "crash" instances (which gives you an idea of the stability and reliability to be expected from them), for engineers to experiment with. No important data is stored there. In the worst case, an instance can be turned down and re-created from scratch if necessary without any impact on user data, and most crash cells are wiped periodically. Most Storage and Database systems also provide shared sandbox-style instances for users to experiment with. The naming is deliberately very different from standard production instances so that users won't get confused.

Data Availability

Once we have ensured our data's integrity and secured it, we can move on to the third layer of the pyramid. Users expect their data to be available when they want to access it. Availability is the guarantee that a user can access their data whenever they need it, defined by specific uptime and latency SLOs that are tailored to the business's core workflows. From an SRE perspective, achieving high availability requires a combination of replication, capacity provisioning, and maintenance—this ensures that the user experience won't suffer from severe latency degradation during operational downtime.

You might not know how the data you're storing is being used. For example, data in Google Cloud Storage might be archives that are rarely accessed or static content used in a high-traffic website. Frequency of access is a poor proxy for estimating the impact of

unavailability, and you can no doubt easily imagine scenarios where unavailability of a rarely accessed piece of data is very impactful. So, for each customer and each data classification, you must determine your latency and availability requirements.

Different companies need different availability for their core workflows. Financial firms generally prioritize consistency because transactions must always be recorded in the correct order: losing a withdrawal or a deposit would result in regulatory problems and customers switching banks. Ecommerce prioritizes availability and latency because if a storefront isn't up, a user can't buy your product and might purchase from a competitor, causing you to lose revenue.

Latency requirements vary greatly between workloads, and different workflows within a company may have very different needs. If you are running a cron job that needs to finish processing analytical data into a daily report for your finance desk, you can potentially tolerate latency on the order of hours. However, a user-facing system such as a payment processor requires subsecond latencies so that customers can make purchases.[3] Architectural strategies such as caching and geographic redundancy can reduce overall load and network costs, and they should be considered when planning where to store your data. Because user-facing transactions must be highly performant, they must be rigorously tested and monitored to ensure they remain fast. From an application or user's perspective, a request timing out is equivalent to the data being unavailable for a short period, so it's important to consider latency while considering availability.

Multiple techniques can improve availability, including:

- Replication or redundancy to provide extra replicas to read from. A simple system design will statically assign jobs to different replicas. A more sophisticated system will dynamically load-balance across replicas.
- Provisioning so that peak usage doesn't overload your storage system.
- Designing systems that allow replicas to support routine software or hardware maintenance without compromising data availability. You don't want to take all the replicas of a piece of data offline at the same time.

Once you know your data and business requirements, you can choose a storage solution based on that information. You need SLOs around both availability and latency; for more details, see "SLOs".

Data Efficiency

We've covered the first three layers of the hierarchy of data needs, and we're at our last stop: data efficiency. If you stored your entire dataset on every machine in a region, and clients read from multiple sources for high availability and low latency, you'd have a high-performance and resilient storage system. It would, however, be prohibitively expensive, and write latency would increase with the number of machines. Throwing money at a storage system may seem like the easiest way to make it resilient and fast, but it's unsustainable at scale. Part of your job is to efficiently store data while delivering resilience and performance, balancing speed, cost, and quality. This isn't easy; it will involve interesting engineering work, including decisions about what hardware and software to purchase, and choosing the right trade-offs based on your specific needs.

Storage costs

There are multiple storage costs to consider when building or buying a storage system: the cost of storing data on HDDs or SSDs, the CPU and RAM required to serve that data and perform maintenance, and the network bandwidth required to copy or move data between regions. You will need to balance these costs against performance, which requires understanding your performance requirements and the options available to you. Storing hot data on SSDs and cold data on HDDs, using memory-based filesystems for temporary storage, and using ferroelectric RAM (FRAM) for writes to reduce latency are all techniques that have been used successfully.

Striping data across regions

One large storage user inside Google has moved from replicating all data across multiple regions to implementing something akin to a redundant array of independent disks (RAID) configuration across regions. Each region stores a subset of the data, like a hard disk in a RAID array storing a subset of the data. The subsets are distributed such that the loss of any single region can be recovered from, like the loss of any single hard disk in a RAID array. Instead of storing two copies of the data (100% overhead), it stores half of the data in one region, half in a second region, and a code block equal in size to half the data in a third region (50% overhead), reducing storage costs by 25%. The data storage layer in each region is stored in Colossus, which similarly uses a RAID-like encoding across multiple machines.

Now that we have described the features a storage system needs to provide and guarantee, let's look at the ways that storage systems can fail, and how we can prevent or mitigate those failures.

What Failure Scenarios Do You Need to Consider?

Users usually don't forgive failures in data management. Companies are generally less forgiving than individuals and will often choose to find another supplier if failures happen too often or are severe enough. Failures result in bad press, loss of user trust, and loss of revenue—sometimes compensation must be paid if an SLA is breached.

Implementing backups without considering the failure scenarios you could be recovering from will result in incomplete coverage or unnecessary expense. Here is a necessarily incomplete list of scenarios to consider.

Operator Error or Malicious Action

Human operators, configuration management systems, and autonomous AI agents are all inherently error-prone and will make mistakes. It's also possible for a human to deliberately and maliciously delete data. Your options for defending against this are:

- Configuring snapshots, backups, or both. When restoring from a snapshot or backup, any writes made between the last known good recovery point (which can be difficult to identify) and the time when the operator error or malicious action occurred will likely be lost. Snapshots share fate with the live data, so it's possible that an event affecting the live data will affect the snapshots, too. Backups are separate copies of the data, so they are less likely to be affected.
- Minimizing human access to your data, particularly unilateral human access, to reduce the risk of operator error or malicious action.

NOTE

Snapshots and backups will be discussed in detail in "Backups and Restores". The quick summary is that a snapshot is a metadata-only operation adding extra references to the underlying data that pin the underlying data and prevent it being overwritten or garbage collected. In contrast, a backup is a fully separate copy of the underlying metadata and data.

Bad Data Written by Applications

All software has bugs, so there is a risk that an application will write bad data. Your options for defending against this are:

- Configuring snapshots, transaction logs (so you can replay writes after fixing the bug), or backups
- Adding application-level checks to detect bad data (e.g., checksums to detect hardware corruption or invariant checks to detect bugs)
- Adding reconciliation pipelines to detect and fix bad data

Note that replication does not mitigate this risk because it automatically propagates corrupt data to all replicas.

Data Destroyed or Corrupted by a Storage System

It's unlikely, but possible, that a bug in a storage system could destroy your data, either at rest or during writes (e.g., deleting a table or writing partial data). How often does hardware failure cause corruption? Research conducted on Facebook's RocksDB key-value storage engine indicated that corruptions were introduced roughly once every three months for each 100 petabytes of data. In 40% of those cases, it was determined that the corruption had already propagated to other replicas.[4]

Your options for defending against this are:

- Configuring snapshots, backups, or both. Snapshots share fate with the live data, so it's possible that the bug affecting the live data will affect the snapshots too. Backups are separate copies of the data, so they are less likely to be affected.
- Writing to two or more storage systems in parallel. These should be in different failure domains (e.g., regions) to minimize shared fate for other failure scenarios.

Data Unavailability or Loss Due to Hardware Failure

Good data written by your application could be corrupted by hardware failure. This is typically only detected when the data is read back and fails validation (hence Google's heavy use of checksumming, as discussed in "Checksums"). The failure of a critical combination of machines or disks will lead to data unavailability or permanent loss in a storage system, similar to losing multiple hard disks in a RAID array. Your options for defending against this are:

- Changing the encoding of your data to tolerate more failures—e.g., changing from RAID5 to RAID6, allowing for high durability with lower storage costs than replication. Note that this is not supported by all storage systems.
- Configuring backups. Snapshots are not effective because they share fate with the live data.

- Replicating your data. This will allow you to serve from a separate failure domain while the hardware is restored. However, it won't protect you from data that was corrupted before it was replicated.

Data Unavailability or Loss Due to Region Failure

In extreme cases, such as a datacenter fire, an entire region may be lost. Your options for defending against this type of failure scenario are:

- Configuring backups, though the destination must be in a different region or offline at a remote storage facility
- Replicating your data to another region

When considering the risk, design a geographic placement strategy to ensure your backups and replicas sit outside the primary location's blast radius. Synchronously replicating a database adds latency to writes and transactions but provides strong guarantees against failure of one region. Replicating files or blobs is typically asynchronous and may take hours. There are additional storage, processing, and networking costs associated with replication that need to be balanced against the increased reliability, particularly for data with a high churn rate.

SLOs

For a detailed treatment of SLOs in general, see Chapter 8. This section discusses SLOs as they pertain specifically to data management.

When designing data storage systems, you must account for three core objectives:

Recovery Point Objective (RPO)

: The maximum acceptable age of data when restoring from a backup

Restore Time Objective (RTO)

: The maximum acceptable time allowed to fully restore the data after a failure

Restore Window Objective (RWO)

: How far back in time you need to restore your data

Figure 16-3 shows where each of these is relevant in the backup plan.

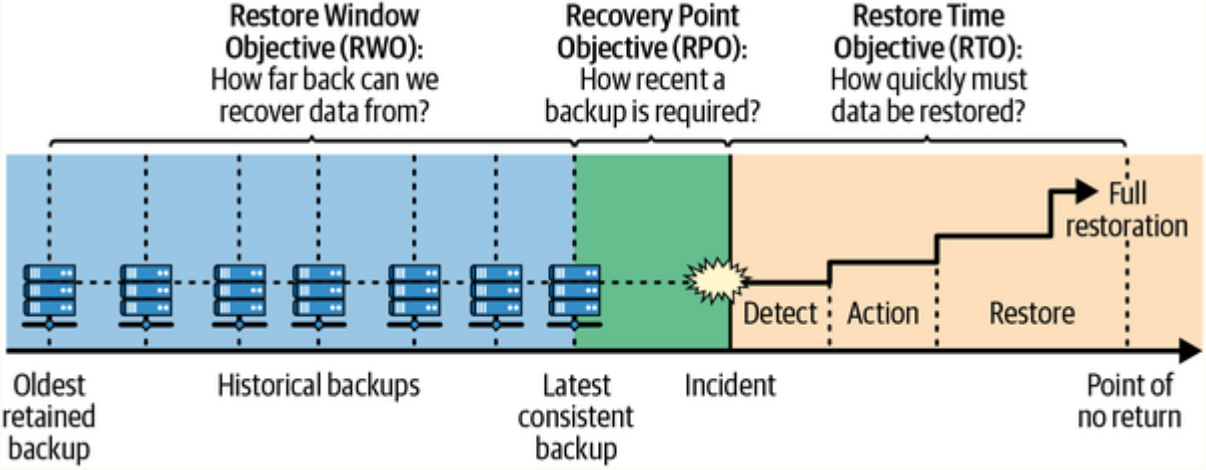


Figure 16-3. Where each objective is relevant in the backup plan

You will build your SLOs on these objectives—for example, "backups for 99% of data sources are within RPO 99% of the time." Don't fall into the trap of evaluating the capabilities of your existing storage systems, then designing and implementing a backup plan that can meet RPO, RTO, and RWO targets that seem achievable based on your current constraints. Rather, your RPO, RTO, and RWO should be considered first, and these should drive the design of your backup strategy. In many cases, they would also drive the design and implementation of your storage strategy.

Recovery Point Objective

How recent must your data be when restoring? Or, looking at it from another perspective, how much business or user data can you afford to lose? RPO defines the maximum amount of time between backups, and thus how much data you're prepared to lose. For example, for secondary reporting systems that aggregate data from other sources, a 24-hour RPO might be perfectly valid. For a banking application, the target might be zero seconds because no amount of data loss is acceptable, even if that target is not 100% achievable.

Deciding on your RPO will determine how frequently you need to take backups, which will limit how long any single backup can take. Limiting the duration of each backup will constrain your choice of backup system and might place an upper limit on the amount of data you can store in a single instance. An upper bound on the size of instances may require you to shard your storage, which may in turn require substantial changes to your application. You can see why deciding on RPO as early as possible is essential—it has a significant impact on many other parts of your system and can greatly constrain your implementation choices.

Restore Time Objective

How quickly do you need to be able to restore your data? The upper bound on acceptable restore time determines your RTO. Note that RTO is a cumulative metric that includes time to detect, time to action, and time to restore. This is ultimately a business decision driven by business needs. Your target RTO will also constrain your implementation choices. For example, restoring from a backup might take a prohibitively long time, so you may need to use snapshots or a hot spare instead. Defining your RTO will help you provision hardware appropriately: you need enough spare capacity for a full restore within the RTO, and if you're talking about restoring 100 terabytes of data, that's a lot of read, write, and network bandwidth—do your systems have enough?

Snapshots are inherently faster to restore than backups because they require only metadata changes rather than data changes, but be aware of the shared fate aspect of snapshots: if the failure mode affects both live data and the snapshot you want to restore from, your snapshot isn't a complete restore option. In a situation like that, you might be able to combine a snapshot restore for the vast majority of data with a targeted backup restore for the data that's damaged in the snapshot. However, you will need to verify that it's possible to identify the corrupted or lost data sufficiently well, that your backup system supports restoring a subset of a backup, and that restoring a subset of a backup will consistently run substantially faster than a full restore. There are a few reasons why this might not be the case, including:

Sequential media constraints

Restoring from a backup on tape requires scanning through the tape to find the files you want. The time to restore depends on the position of the data on the tape; it will be faster if the data is at the start of the tape, but if it's near the end it won't be any faster than reading all the data off the tape. You might also need to transport the tape from off-site, adding to the delay.

Granularity limits

If you back up an entire SQL database, your minimum restore granularity is likely to be a full table, even if you need only a single row from that table.

Archive layouts

If you tar up multiple files and archive the resulting tar file, you will need to read through that file until you find the file you want. Zip files support seeking to the right file, but if your zip file is in cloud storage you will most likely have to transfer the entire file to local storage anyway.

Restore Window Objective (RWO)

How far back in time do you need to be able to restore your data? That window of available restores determines your RWO. Data loss, and particularly data corruption, frequently go undetected until something tries and fails to read the data. For example, if you have a policy of daily backups with a two-backup retention window, then you can't recover from corruption from three days ago. Conversely, a retention policy of 14 days allows recovery from any day in that period, with the caveat that up to 24 hours of data could be lost. Why 24 hours? If backups are taken at midnight and corrupted data is written at 23:50, then the last good backup is almost 24 hours old.

Your RWO must be long enough to meet business requirements, while being short enough to satisfy data retention and deletion requirements (see "Legal and Ethical Considerations") and keep costs acceptable. Some workloads will naturally read all data in a dataset and quickly detect data loss or corruption.

Replication

Data replication is a strategic business decision that balances cost against improved performance and availability, by giving you multiple regions to serve from so a service can survive the loss of one region. However, replication cannot be your sole recovery mechanism because of two well-known failure modes:

Corruption propagation

If corrupt data is written to a primary instance, it immediately replicates to all downstream copies, leaving no pristine state to restore from.

State-change replication

Replication offers zero protection against accidental human error, application bugs, or malicious deletion, because the delete command happily replicates everywhere.

To recover, you need a way to (partially) revert the corrupted data to a known good state, and for that you need snapshots or backups. If you are running a data analysis job on intermediate data, that intermediate data can be regenerated if lost, and only one copy may be necessary. If you're storing data on credit card transactions, you cannot lose this data, for both regulatory and customer trust reasons.

Considerations When Deciding Whether to Use Replication

If you choose to use replication, you will need to consider RPO, RTO, and RWO, as described in "SLOs". Determining how many replicas your data needs is only the first step; you need to consider the maximum replication delay that still meets business needs. Synchronous replication makes recovery simpler, but it adds write latency to ensure consistency and also has the downside of replicating bad data synchronously. Asynchronous replication is fast, but it might be delayed for a long time, especially for hot data. This applies particularly to failovers: if your replication is five minutes behind, triggering a failover is effectively performing a hard reset to the state of the world five minutes ago. If that data loss isn't acceptable, your replicas are providing an expensive false sense of security, and you should really be looking at a strongly consistent replication strategy.

For most mission-critical storage systems, the industry standard has been to store three replicas of the data, generally at least in different datacenters, but preferably in different cloud regions so you are guarded against natural disasters, utility failures, geopolitics, etc. If one datacenter goes offline, the business will remain operational with near-zero RPO.

Many workloads (e.g., online shopping) have far more reads than writes. In these cases, read-only replicas are an effective way to reduce load on the writable replicas. These replicas can also be deployed closer to users, so that only writes need to be sent to another region while reads are served locally.

Be sure you thoroughly understand the details of how a storage system scales with each additional replica before deployment, to ensure you do not hit an unexpected performance cliff during high-growth phases.

Evaluating a storage platform requires analyzing its replication architecture and observing how it behaves under realistic production workloads. There are two main configurations:

Single-leader topologies

The simplest replication model, featuring a single primary and multiple followers. Because all write operations are routed through the single leader, the system enforces strict ordering constraints out of the box.

Multileader topologies

A more flexible, resilient, and scalable configuration featuring multiple primaries, potentially paired with multiple followers. This approach is well suited to geo-

graphically distributed deployments because clients can write directly to their local replica.

With multileader topologies, there must be a method of conflict resolution to handle overlapping writes to different leaders. The existence of conflicts isn't always reported to writers, and "last write wins" semantics may not be appropriate for your application. Ensure that you and your application developers understand the semantics of conflict resolution.

Another important factor is whether you want to make replication synchronous or asynchronous. Synchronous replication is much easier to reason about because a successful write will be visible on other replicas, but write latency is higher. Asynchronous replication offers much lower write latency, at the cost of a more difficult consistency model because a successful write may not be visible everywhere immediately.

For asynchronous replication, determining an upper bound for latency is important—what is the maximum replication latency that meets your application's needs, and how do you ensure that replication latency stays below that limit? If the leader of your data storage system is sharded and your replication is asynchronous, you need telemetry into the replication latency of different follower shards, because the application may need to handle different replication latencies depending on the shard.

Is replication guaranteed or best effort? In a best-effort scenario, your system attempts replication and abandons it if it's unsuccessful; therefore, it's potentially lossy. If you operate under best-effort semantics, you will need processes to handle divergent replicas. Divergent replicas are hard to detect and extremely hard to repair, particularly when the divergence cascades down to the application layer.

Lastly, when changing replication strategies, don't forget to plan for any additional compute, networking, and storage requirements, as these could impact replication performance.

Dealing with the Loss of a Replica

Replica loss will happen eventually, and you need to plan for the many different ways this can occur. As we've seen, there are two main topologies to consider, single leader and multiple leader, and we can lose either a follower or a leader in each. (Leaderless topologies like Cassandra deal with loss of a replica transparently.)

Single-leader topologies

If a follower in a single-leader topology is temporarily unavailable but will recover, waiting can be a viable strategy. However, you will need enough storage on the leader to store the growing backlog of data that needs to be replicated, and there will be substantial load on the leader when the replica returns.

In the event that you lose a follower permanently, there should be a well-documented process for replacement. In many systems, that replacement can be performed automatically, but there are a few things you'll want to consider before enabling this feature:

- Ensure that applications will automatically switch to the new follower, and that there is sufficient capacity on all nodes to handle one replica being down for an extended period. Because asynchronous replication can result in data gaps during a failover, ensure that your applications can handle skipped or delayed data.
- If you have only one follower (common with leader-follower database systems), is it better for applications reading from it to switch to the leader, or should they wait for the new follower, to avoid adding load to the leader? Ensure that the leader can handle the extra traffic. SREs should implement load shedding so that low-priority clients are limited during these peak periods.

If you lose the leader, on the other hand, waiting is unlikely to be a viable strategy because no writes can be performed. Promoting a follower to leader should be an automated process or a well-documented and well-tested manual process.

If replication is synchronous, it's unlikely that any writes were lost. However, if replication is asynchronous, some writes may not yet have replicated. Consider the following:

- Do you have visibility into how delayed replication was so that you can calculate an upper bound on lost writes?
- Is it possible to replay those lost writes from application logs or other sources?
- How will your applications handle lost writes?
- Is automatic promotion of a follower to leader an option? If so, will your applications switch to the new leader once it has been promoted, or will they need to be reconfigured or restarted?

Multileader topologies

The concerns related to losing a follower in a multileader topology are the same as those in a single-leader topology. However, the situation is different when it comes to losing a leader. Generally, replacing a leader is less urgent because the remaining leaders can

continue to accept writes. Understanding whether writes were lost in asynchronous systems is potentially much harder, though, because replication delay can be highly variable, and you may be dealing with a fully connected graph with *N* nodes.

If writes have propagated only to a subset of leaders, it may be possible to restore those writes to the leaders that do not have them. Beware of split-brain scenarios caused, for example, by a network link being down, as this may require manual determination of which writes should be retained. Ideally, the software will provide tooling to support this, because it's complex; the process needs to write to a given replica without that write being replicated to all other replicas.

Backups and Restores

People often focus on backups without sufficient consideration for restores. Everybody cares about restores on the day they need to restore the data and service to users, but the process and outcomes are often untested. You need to figure out an appropriate backup schedule and process, ensuring that your backups don't share fate with the data.

Snapshots Versus Backups

It's important to understand the difference between *snapshots* and *backups*. Many storage and database systems provide *copy-on-write snapshots*, which generally only require changes to metadata to preserve data as it was at a certain point in time.

However, snapshots partially share fate with the data they're a snapshot of: if a live file is lost through data corruption, it will also be lost in all snapshots it's present in. Conversely, snapshots remain unaffected by some other types of failures, such as an application bug writing bad data or an operator error deleting a file or directory. Because snapshot generation is quite cheap, teams can take snapshots frequently. They are particularly helpful as insurance before making a risky change, because you can quickly and easily roll back to the snapshot you took. This will still involve the loss of mutations made after the snapshot was taken, but it's better than complete loss of the data.

Backups are separate physical copies of the data, not logical copies like snapshots. Taken correctly, they should not share fate at all with the original data. There is a substantial cost to taking them, however: data needs to be read from the original location, transferred across the network, and written to the new location. (For strategies to balance backup costs against restoration performance and reliability, see "Full, Incremental, or Differential?".)

The greater the separation between the original and the backup, the better: a backup on the same hard disk as the original data saves you if you overwrite the original data but doesn't save you from hard disk failure. A "3-2-1" backup strategy is recommended:

3 copies

Maintain three distinct instances of your data (one primary production copy and at least two backups).

2 media types

Store the backups on two different physical media types to protect against systemic hardware defects.

1 off-site location

Keep at least one backup copy off-site or in a geographically distinct cloud service, taking local data residency and regulatory mandates into account.

The credentials required to modify or delete a backup should be highly restricted, and ideally anything that can write to live data will have zero administrative access to backups to prevent the same bug or malicious action affecting the backups as well.

In practice, the most effective disaster recovery strategies often combine both mechanisms to gain the benefits of each. Snapshots are cheap and quick to take, so you can have many of them, and they are quick to restore, allowing for short RTOs. Backups are more expensive and slower both to create and to restore, but they do not share fate with the original data, as snapshots do. Having both gives you the option to quickly restore a recent snapshot when that's the appropriate response to data loss or, when necessary, to restore from a backup that's unlikely to have suffered the same data loss. The right mix of snapshots and backups will be determined by your RPO, RTO, and RWO, and their associated SLOs.

Point-in-Time Recovery

Point-in-time recovery (PITR) allows you to restore a system to an arbitrarily chosen point in time, rather than the time you last took a snapshot or backup. It's typically implemented by taking a backup and then storing a log of all writes applied after the backup. Recovery is performed by restoring the backup and replaying the log of writes up to the specified time, giving you a very small RPO.

The log of writes is expensive to store, particularly if hot data is overwritten frequently, because you store every write rather than just the most recent one. Restores can also be expensive and slow because writes are replayed. However, provided the organization can tolerate the accompanying storage costs and restoration latency penalties, PITR provides an exceptional safety net for mission-critical data (e.g., financial data).

Full, Incremental, or Differential?

Depending on the system you are backing up, you may have the option of full, incremental, or differential backups. Let's discuss the pros and cons of each to help you make an informed decision about which approach to choose.

Full backups

With full backups, you take an entire new copy of the original data every time you take a backup.

Pros:

- Very simple.
- Restoring requires only one backup archive, unlike with incremental and differential backups.
- Each backup archive is entirely independent of other backups. No data is shared, so corruption or loss of one backup cannot affect others.

Cons:

- Very large storage overheads. As well as sufficient compute, I/O, and network resources to perform the backups, to store *N* backups, you need *N* times the storage capacity of your original data.
- Speed. Full backups are slow because of the sheer amount of data to be copied. This might limit the backup frequency, which in turn limits your RPO, RTO, and RWO.

Incremental backups

With an incremental approach, you periodically take one full backup, then you take *N* incremental backups, each containing only the changes since the previous full or incremental backup. This is conceptually similar to version control, where you start with an empty file and commit the diffs between versions.

Pros:

- This is the backup option with the lowest overhead, because you're only backing up changes since the previous backup. It requires less storage, less I/O, less network, and less compute—but only if there is an efficient way to identify the changes that need to be backed up (see cons).
- Incremental backups are faster (after the initial backup), so you can take them more frequently, enabling a more recent RPO.

Cons:

- Restoration is much harder and slower because you need the latest full backup plus all incremental backups since then.
- Loss of or damage to an incremental backup affects every subsequent backup until the next full backup.
- Not all systems have an efficient way to identify the changes that need to be backed up. For example, incremental backups work well on Unix filesystems, where last modification time tells you whether a file needs to be backed up. They don't work as well on traditional databases, where there isn't a native timestamp to identify modified rows that need backing up.
- Although the incremental backups are fast and cheap, you need to provision enough resources for the periodic full backup, including enough destination storage for one full backup, all the incremental backups, plus the new full backup—or more, depending on your RPO.

Differential backups

With this approach, you periodically take one full backup, then every subsequent differential backup contains everything that has changed since the last full backup. Figure 16-4 compares this to the incremental approach.

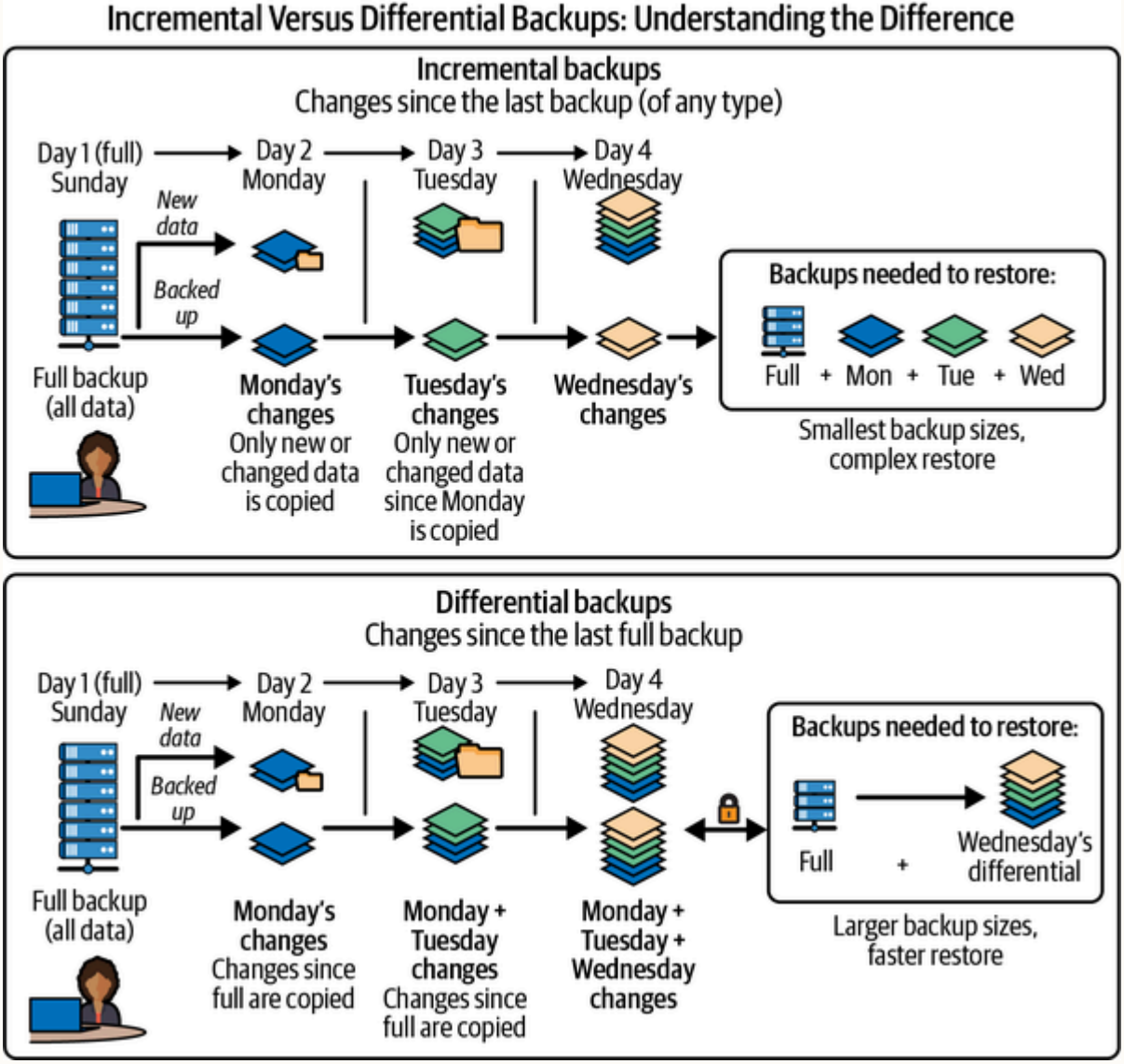


Figure 16-4. The differences between incremental and differential backups

Pros:

- Lower compute, network, and storage overheads than full backups.
- Restores require only the full backup plus one differential backup, making them simpler and cheaper than the incremental approach and reducing the risk of shared fate. Loss of a differential backup affects only your ability to restore that backup; it does not affect your ability to restore from a later backup.
- You can take the differential backups more frequently than full backups.

Cons:

- Provisioning is similar to incremental backups, except that each differential backup will be more expensive than the last, until you reset with a full backup. When your data changes a lot, a differential backup can be nearly as large as a full backup.
- Not all systems have an efficient way to identify the changes that need to be backed up.

Smart or Simple?

Another consideration is whether you should choose to make simple backups that might be more reliable but more expensive or smart backups that might be less reliable but cheaper.

Smart backups minimize ingestion windows and storage overhead. However, they can be less reliable because the smarter the backup, the more there is to go wrong and thus the more likely it is that the backup will be unrestorable. Additionally, the smarter the backup, the more there is that needs to go right during the restore, even when the backup was taken correctly.

Compare two ways of backing up a SQL database: copying the raw database files versus exporting the entire table via a SQL query. The former is smarter because you're not putting a spiky load on the database and you don't need intermediate storage for the backup. However, the latter is simpler to reason about because there is no risk of backing up inconsistent data by reading files while the database is writing to them. The downside is a significant load spike on the database.

For restoration, the copy approach has more potential failure modes because if the database files are corrupt you've probably lost the entire backup. Another possible failure mode is that they'll work only with the same database software, and even potentially only the same version of that database. Restoring from the SQL export is more reliable since if something is wrong with the backup, you can try to edit it, split it into chunks, or restore it to different database software if necessary.

When you're designing your backups, it can be tempting to use the cutting-edge new feature—but it's important to consider the new failure modes it brings and how or even if you can recover from those failure modes. We're not saying never use that new feature, just that you should give it due thought and test it frequently.

Restore Testing

All too often, teams don't test restores and only discover during an active crisis that a recovery path is broken, infrastructure resources are insufficient, or operational drift has rendered historical procedures obsolete. The solution is to test restores on a regular basis, ideally using automated tests rather than manual ones, with the aim that a manual restore just requires running the automated restore process with some different flags. This will likely require extra machine resources, but if you don't have the resources to do a test restore, do you have the resources to do a real restore? Sharding your backups will

reduce the resources required for testing and provide the additional benefit that restores will be faster if they're ever needed for real. Automated restore testing, with proper monitoring and alerting, will help ensure that you can meet your RTO and will catch regressions (see Chapter 14).

You need to consider different types of restores and ensure that your restore processes and testing cover each type. For example:

- Restores as part of normal business operations—e.g., restoring a nightly backup to a separate instance to run report generation against so that load doesn't affect serving user requests
- Partial restores requested by a user—e.g., *I deleted all of my email; can you get it back for me please?*
- Partial or complete restores in response to an application writing bad data
- Partial or complete restores in response to hardware failure, ranging from a single bad hard disk to loss of a datacenter.

In addition to automated restores, the manual actions needed to initiate a restore must be documented, tested, and practiced to the same level as you practice other incident responses. Restore testing is an important part of your backup plan (discussed in the following section).

Let's imagine a hypothetical strategy for backing up YouTube:

1. Generate a list of all videos.
2. Copy the files containing the videos.
3. Copy all the video metadata.

While this seems logical, there's an inherent race condition. Any videos that are uploaded between steps 1 and 3 will have their metadata backed up but not the video files themselves, because they won't have been included in the list of video files to copy. In other words, if we restore this backup, some videos will have metadata but won't have backing files to play. The bug here is that all the metadata is backed up, rather than it being filtered to match the video files being backed up.

There are two solutions to identifying bugs like this, and both are important. The first solution is human analysis looking for flaws in the design so they can be fixed. Google SRE has adopted STPA for this (as discussed in Chapter 12). The second solution is to have the application validate restored data and flag discrepancies for investigation, and include that application-level validation in your automated restore testing. In the YouTube example, a process that validates video metadata and backing video files would

be useful to have anyway. Wiring it up to run against a test restore wouldn't require much effort but would bring significant benefits. This type of application-level validation will not only help improve backup and restore processes; it will also identify opportunities to improve the application itself to reduce internal inconsistencies (even temporary ones).

Restores are a key component of disaster recovery, and as such are essential to a functional business continuity plan. Give them the time and attention they deserve and require.

Creating a Backup Plan

Many people set up backups ad hoc rather than planning them. While this is better than not having backups at all, with an ad hoc approach it's easy to wind up with gaps in your backup coverage—for example, you might be missing one dataset entirely or have snapshots but not offline backups. Proper planning helps you avoid these gaps. It also helps ensure you aren't making too many backups, whether by backing up to multiple systems without getting extra benefit from the additional backups, backing up too frequently, or keeping your backups for too long (depending on your business needs, backups from a year ago might not be useful). Proper planning for backups will help keep your backups efficient and properly provisioned. This section provides guidance on how to write a backup plan, as well as a few thoughts on implementation.

NOTE

To avoid cumbersome repetition, in this section, we use *backups* as a collective term for physical backups, snapshots, PITR, replication, and other recovery mechanisms described in this chapter.

The Planning Framework

Developing a resilient recovery strategy requires following a structured five-stage lifecycle:

Data discovery and classification

: Identify all organizational data assets and categorize them by business criticality.

SLO definition

Establish explicit RTOs, RPOs, RWOs, and corresponding SLOs for each data classification.

Architecture

Determine the appropriate technical recovery architecture for each tier based on its defined SLOs.

Validation design

Design and implement automated restore testing and verification pipelines across all datasets and storage systems.

Governance

Document the overall strategy and secure formal sign-off from key organizational stakeholders such as the CTO, CFO, and COO.

Keep Business Needs in Mind at All Times

Your job is to design and implement a backup strategy that meets business needs and contributes to business continuity goals. An insufficient backup strategy places the business at risk. Overly frequent backups are an unnecessary expense for the business. Backups that are retained for too long might create legal risk. Some data should be retained indefinitely if practical—e.g., source code or legal documents. Some data is irreplaceable, some can be regenerated easily, and most falls somewhere in between. You need to balance all of these competing requirements and regularly review your backup strategy to ensure it stays relevant.

Classify Your Data Sources

The simplest strategy for implementing backups is to apply a one-size-fits-all approach, but that will nearly always result in some data sources being backed up at the wrong frequency or in a way that's unsuitable. At the other end of the spectrum is having a completely custom backup configuration for every data source, which is fine if you have only a few but quickly becomes hard (or impossible) to manage. Somewhere in the middle is the sweet spot, and often the best way to achieve that is to classify your data sources into different buckets and use the same backup configuration for every data source in the same bucket.

To help you determine what your buckets should be, we recommend building a catalog of data sources. Such a catalog has many more uses—e.g., compliance audits—and may

already exist for other reasons. There is some overlap with data classification (see "Data Classification"), so it often helps to start with that. Defining buckets first and classifying datasets into them might not result in the best classifications; a more useful approach is to group similar data together and then determine an appropriate classification for it. Once you have the catalog, give each data source some labels, based on your business requirements. Examples might include *user-provided*, *frequently updated*, *derived*, *legally sensitive*, *personally identifiable*, and *low-value*. (Low-value? Seriously? Yes, some of your data is likely low-value compared to other data, and it's worthwhile taking that into account when creating your backup plan.) You can then use those labels to bucket your data sources. Depending on how many buckets that results in, you might need to combine buckets to keep the number manageable.

Set the Objectives for Each Bucket

Now that you have buckets, you need to set an RPO, RTO, and RWO for each bucket; these determine the intent of the backup configuration. These two requirements, for example, will result in very different backup configurations:

- We can tolerate losing up to 24 hours of data, and the business will be okay if we can restore within 12 hours.
- We can't tolerate more than 5 minutes of data loss because we buffer 10 minutes of data for replay, and we need the restore to happen in 3 minutes so that we have 2 minutes to replay the buffer before its contents start being discarded.

It should also be clear that how you respond in each of those scenarios will be very different: manual response will be fine for the first, but the response to the second absolutely must be initiated and driven by software because a human can't respond fast enough to prevent data loss.

Keep in mind that some data shouldn't be backed up—the business value might be too low to justify backups, or regenerating the data might be cheaper than backing it up—and make sure you have an explicit SLO for data without backups.

Decide on the Backup Configuration for Each Bucket

The discussion in "Backups and Restores" can help guide your backup configuration decisions. Key decisions include:

Snapshots or backups?

Snapshots are fast, lightweight, and can be frequent, but they don't protect against all failures, whereas backups are slow and add a substantial load on the storage systems but protect against many different types of failures. You will probably need both.

Full, incremental, or differential?

You need to weigh the trade-offs between the options (if your storage system supports all of them) and decide which best meets your needs. Also consider PITR backups.

Smart or simple?

Consider how the backup process can fail, and whether the risk of failure is justified by the benefits. Pay particular attention to whether there can be correlated failures where both backups and live data are affected by the same problem.

Decide How You Will Perform Restore Testing

We've already discussed restore testing. This is a quick reminder that backups alone don't provide any business value; restores as part of recovering the service do. Consequently, restore testing is essential and needs to be a first-class part of any backup and recovery plan. In some cases, changes will be required in storage systems or in applications to make restore testing viable, and it's highly likely that you will need to argue in support of those changes and convince people of the necessity of investing in this testing.

Manual restore testing is better than no restore testing. Automated restore testing, with alerting on restore failure or RTO failure (and automatic validations that the recovered service is functional and consistent), is the gold standard.

Write the Backup Plan

It is very important that all stakeholders, including leadership, the legal team, and your engineering peers, have a chance to review and understand your backup plan. Documentation of all relevant aspects of the plan, including the reasoning behind specific elements, is required so that everyone can understand the decisions. In addition, the plan must be updated regularly to ensure that it continues to meet business needs as things change over time.

It's best to annotate your data sources with the backup plans and SLOs so that it's clear to users what they can expect. In particular, it should be easy to determine whether the SLO is sufficient for new use cases for any piece of data.

Implement the Backup Plan

Once you have approved backup policies, they need to be applied to your data sources. This is a substantial change to your operating environment: it consumes resources on the storage system and on the backup system. It may be tempting to back up everything at once, but it's unlikely that your backup system is provisioned for this, and the extra load on your storage systems could trigger an outage. It's better to test the backups and stagger the full deployment, similar to other deployments (for more information, see "Change Management"). During normal operation, you should stagger the backup schedules—a common antipattern is to run every backup at midnight, overloading either the backup sources or the backup destinations.

Another common failure mode is to perform a one-off sweep through all your data sources as part of setting up backups, but not to set up backups for new data sources. This can happen if creating new data sources is something your users are empowered to do but there isn't a standard creation process that includes configuring backups. Similarly, if there isn't sufficient guidance to enable users to choose the correct backup strategy, users might defer making the decision for a long time or make a bad decision that doesn't meet their needs. Configuring backups and provisioning backup resources for new data sources must be a standard (or automatic) part of setting them up. In many cases, you will need your internal users (including you) to supply some metadata for each data source that you can use to classify it. Ideally, the process of creating new data sources will include this metadata or will directly configure the backup bucket.

Creating a Data Recovery Policy

Despite your best precautions, eventually some data will need to be restored or recovered. Maybe a bug in a piece of software writes bad data, an operator makes a mistake (though the system would ideally be robust against operator errors), or a hardware failure causes data to be lost or corrupted. If you have been testing your restore process, then your restore should work, and in the majority of data loss events that will be sufficient. When a restore is insufficient, you will transition to recovery. You should have a policy covering data restoration, data recovery, and data loss. This policy should be written well in advance of this type of event, reviewed and approved by leadership, and communicated to users so they know what to expect. Setting user expectations is very

important: if they know that data will be declared lost in some scenarios, they are more likely to implement application-level safeguards than if they expect heroics from you.

THE DATA RECOVERY PROCESS

Failures eventually happen, even if the probability of failure is minuscule. A *data restore* is the routine, well-rehearsed process of reinstating data directly from a backup to recover from typical operational incidents. In contrast, *data recovery* refers to the extraordinary tactics and reconstruction efforts employed to salvage or rebuild data only after standard restore mechanisms have failed or proven insufficient. Your product should have a data recovery process in place, as you really don't want to be inventing that process the first time you need it. This could be as simple as dropping a replica and re-replicating, or you might use tooling that will reconstruct data from another source. Sometimes you will be able to restore at leisure, but sometimes there will be a hard deadline—e.g., it's 11:10 p.m. and you need the data back for the daily pipelines that start at midnight. The classification you performed on your data will help incident responders determine the importance of the data, the urgency of restoration, and whether the best course of action is simply to declare it lost.

Google's internal Bigtable service uses the following data recovery policy:

> *Infra Bigtable will perform data recovery to the fullest extent possible before declaring data loss, however data integrity is more important than data recovery, therefore we will not take any action that violates data integrity. When data recovery is complete we must be able to guarantee that a human has not compromised our users' data. Any exceptions to this policy cannot be made unilaterally by Infra Bigtable and so will require appropriate approvals, with the exact approvals depending on circumstance, e.g., Dev or SRE VP approval, Customer approval, Legal approval.*

This information is followed by a much longer explanation of what this means for Infra Bigtable SRE and Dev, and what it means for users. The sentence about humans compromising data is expanded upon in the longer explanation. To summarize, our source code is peer reviewed and our software is verifiably built and deployed, so we can give strong guarantees about data integrity and provenance. If humans are manually changing data in the recovery process, we lose those guarantees. Therefore, we prohibit manually changing data, even at the cost of losing that data, unless signed off on by people with the authority to make that decision and trade-off.

Considerations to keep in mind when writing your policy include:

- Can you repair the data rather than restoring it (e.g., by rerunning a pipeline or by running specialized tooling designed for repair)? Your SLOs will be essential in determining whether repairing or restoring is the better response.
- Can you perform a partial restore?

- Can you minimize the loss in the data that has been written since the last backup was taken? E.g., if a bitflip has corrupted part of a file, can you extract the data before and after the bitflip? Google checksums individual records inside files to enable extraction of uncorrupted records from a corrupted file.
- If some data loss is unavoidable, how will the decision be made of which data to lose? E.g., if the choice is between losing some rows in a database completely versus rolling the entire database back to the last backup, who makes that decision, and what information will they need to make an informed decision?
- Does restoring a backup require taking the data source offline for the duration of the restore? Is that downtime acceptable?
- Is restoring the data more important than maintaining the security of the data? E.g., is it acceptable for engineers to examine the contents of the corrupted data to help understand the corruption?
- Can you debug or repair the corruption without breaking any laws or contractual agreements? E.g., if data needs to be moved to someone's workstation for repair, does that violate guarantees about data locality or data provenance?
- Someone will eventually want to override the policy, and generally they will have a good reason—for example, if the value of the data that would be lost exceeds the legal or other costs of restoring it. Who has the authority to override the policy? Can the owner of the data agree that the policy can be bypassed, or does it require approval from legal and a VP or similar? Who will be available to provide that approval outside of business hours? Whose responsibility is it to obtain the approval—the owner of the data, the team running the storage service, or someone else entirely?
- Is engaging outside assistance, such as a data recovery specialist, an acceptable and viable option? If it is, does that affect how you handle the event? E.g., do you minimize writes to reduce the risk of overwriting data on hard disks?
- Is your data recovery tooling good enough that you can honestly tell your users *we did everything we could*, or does the tooling need improvement?

Conclusion

Data management is a complex area involving considerably more architectural layers, strategy, and discipline than might initially be expected. Effective data management is far more than a technical checklist; it is a fundamental commitment to the users who en-

trust their most sensitive information to our systems. As we have explored in this chapter, this responsibility spans a spectrum from navigating complex legal jurisdictions to engineering systems that satisfy the foundational tiers of the hierarchy of data needs: integrity, security, availability, and efficiency. Integrating these requirements into the design minimizes the cost of remediation. If someone is sailing from Ireland to New York, they want to find out they're off course as soon as possible, rather than when they land in Florida.

The choice of storage system will significantly affect the implementation of the software using it, and changing storage systems is a long and expensive undertaking. Your choice of storage system needs to be appropriate for your business requirements, and it must be capable of scaling to accommodate growth or have a clear and well-supported migration path to a system that will scale. The backup, replication, and recovery strategies supported by a storage system must be first-class considerations when evaluating it.

The core of a robust strategy lies in the shift from ad hoc reactive measures to proactive, SLO-driven planning. By defining clear Recovery Point Objectives, Restore Time Objectives, and Restore Window Objectives, teams can design architectures that align with actual business needs rather than arbitrary technical limits.

Designing these systems requires balancing the day-to-day efficiency of optimized, "smart" backups against the resilient simplicity needed during a high-pressure restore. Operationally, teams must prepare for both routine, automated restores and the extraordinary engineering tactics of data recovery when standard safety nets fail. Even the most sophisticated backup plan is only as reliable as its last successful test.

Ultimately, failures are a statistical certainty at scale. Whether facing silent data corruption, operator error, or a catastrophic regional failure, the resilience of your service depends on a well-documented and leadership-approved data recovery strategy. By treating data management as a fundamental aspect of system design from the outset, organizations can safeguard their most valuable assets, maintain user trust, and ensure that they are never in the position of closing the stable door after the horse has bolted.

1 For an analysis of the costs of a data breach, see Estefania Vergara Cobos and Selcen Cakir, *A Review of the Economic Costs of Cyber Incidents* (World Bank, 2024). For information about the frequency of data breaches, see for example Verizon's Data Breach Investigations Reports.

2 The burden of generating test data (including images and videos) has been greatly reduced by AI.

3 Deloitte Digital, "Milliseconds Make Millions", 2020.

[4] Siying Dong et al., "Evolution of Development Priorities in Key-Value Stores Serving Large-Scale Applications: The RocksDB Experience", in *Proceedings of the 19th USENIX Conference on File and Storage Technologies* (2021).

Previous chapter
15. Capacity Planning

Next chapter
V. Resilient Products and Emergent Reliability